DEMO USING AGENT SKILL MODE (tested with GPT 5.3 codex)

Enable "Auto-approve" to avoid many confirmations

Enable SKILL

Selezionare database Bergamo_2026

--- restart SSMS to have predictable number of confirmations ---

--- after confirmation for Skill + confirmation ofr LLM, back to slides ---

PROMPT 1 – QUERY STORE PERFORMANCE ANALYSIS USING SKILL

```
Use the sql-performance-investigator skill.

Investigate the SQL performance problem occurred on local machine (SQL machine time) in the database [Bergamo_2026].
Investigate the issue using Query Store.
```

OUTPUT:

Query Store Investigation Completed (`[Bergamo_2026]`)

I used the `sql-performance-investigator` workflow and ran a read-only Query Store analysis.

1) Initial symptom and dominant resource

- **Dominant metric: CPU** (with matching read growth).
- Top contributor:
  - `query_id = 1` (`dbo.GiveNotes`)
  - **97.92%** of total Query Store CPU (`1,382,706.82 ms` of `1,412,`~k ms)
  - `11,517` executions
  - Query text pattern: `WHERE ID BETWEEN @N2 AND @N1 GROUP BY Name`

2) Plan discovery and comparison

Discovered plan IDs for `query_id = 1`: **`1, 6, 11, 14`**

Per-plan weighted averages (regular executions):

- `plan_id 1`: `avg_cpu 0.26 ms`, `avg_reads 2.23`
- `plan_id 6`: `avg_cpu 9.98 ms`, `avg_reads 66.09`
- `plan_id 11`: `avg_cpu 83.31 ms`, `avg_reads 1,773.65`
- `plan_id 14`: `avg_cpu 755.07 ms`, `avg_reads 11,436.20`

Execution plan XML evidence:

- All plans use **`Clustered Index Scan`** on `PK_Notes` (`id2`), not a seek on `ID`.
- Estimated rows rise sharply across plans (1 → 2,046 → 262,142 → 1,048,570).

3) Same-plan degradation evidence

Same plan degradation is confirmed on **`plan_id 14`**:

- `avg_logical_reads` grows from **4,224** to **34,738** in later intervals.
- `avg_cpu_ms` grows from **280.39** to **2,564.88**.
- This confirms degradation is **not only** a plan-switch effect.

4) Strongest root-cause hypothesis (validated)

Confirmed evidence

- `dbo.Notes` has ~**8,388,606 rows**.
- Only index is clustered PK on `id2`; predicate uses `ID`.
- Concurrent heavy writer query (`query_id = 3`) is:
  - `INSERT INTO Notes ... FROM Notes`
  - This causes rapid table growth and rising per-execution work.
- Waits for `query_id=1` are led by Parallelism, but waits alone cannot explain the large logical read increase.

Supported diagnosis

Primary issue is **work amplification + non-optimal access path**:

1. Read query filters on `ID` but scans clustered index on `id2`.
2. Data volume grows rapidly due self-insert workload.
3. More rows scanned per execution drives reads/CPU up dramatically.

This is **not primarily parameter sniffing** and not pure plan regression.

5) Corrective action (not executed)

A focused index candidate for the read pattern:

```
USE [Bergamo_2026];
GO
CREATE NONCLUSTERED INDEX IX_Notes_ID_INCLUDE_NAME
ON dbo.Notes (ID)
INCLUDE (NAME);
GO
```

Expected effect:

- Enables range seek on `ID` instead of clustered scan on `id2`.
- Reduces logical reads and CPU for `dbo.GiveNotes`.

Trade-off:

- Extra write/storage overhead (important here because insert workload is heavy).

6) Validation plan (Query Store, before/after)

Track for `query_id = 1`:

- execution count
- avg/total CPU
- avg/total duration
- avg/total logical reads
- plan IDs and interval variability

Success signal:

- sustained drop in logical reads and CPU under comparable interval workload.